

Coding & Solution

Project: Auto Ticket Classification using Flow Design

Feature: Flow-based Ticket Classification and Routing

The solution is written in Python. It trains a TF-IDF + Logistic Regression model on sample tickets, predicts the category, detects priority from urgent keywords, and uses a confidence check to decide between auto-routing and manual review.

Source Code (ticket_flow.py)

```
"""Auto Ticket Classification using Flow Design.

Flow: intake -> preprocess -> classify -> decide -> route / manual review -> notify
"""
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

CONFIDENCE_THRESHOLD = 0.50
TEAM_ROUTES = {
    "Billing": "Billing Team",
    "Technical": "Tech Support Team",
    "Account": "Account Services Team",
    "General": "General Helpdesk",
}
URGENT_WORDS = {"urgent", "asap", "immediately", "down", "crash", "crashed", "outage", "blocked"}

TRAIN_DATA = [
    ("I was charged twice for my subscription", "Billing"),
    ("Need a refund for the last invoice", "Billing"),
    ("Payment failed but money was deducted", "Billing"),
    ("Wrong amount shown on my bill", "Billing"),
    ("App crashes when I open the dashboard", "Technical"),
    ("Server is down and pages are not loading", "Technical"),
    ("Getting error 500 while uploading a file", "Technical"),
    ("Software is very slow after the update", "Technical"),
    ("I forgot my password and cannot log in", "Account"),
    ("Please change the email address on my account", "Account"),
    ("My account is locked after many attempts", "Account"),
    ("Need to delete my profile permanently", "Account"),
    ("What are your working hours", "General"),
    ("How do I contact the sales team", "General"),
    ("Do you have a mobile application", "General"),
    ("Where can I find the user guide", "General"),
]

def preprocess(text: str) -> str:
    """Lowercase and remove punctuation and extra spaces."""
    text = re.sub(r"[^a-z0-9\s]", " ", text.lower())
    return re.sub(r"\s+", " ", text).strip()

def build_model():
    texts = [preprocess(t) for t, _ in TRAIN_DATA]
    labels = [c for _, c in TRAIN_DATA]
    model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)),
                          LogisticRegression(C=10, max_iter=1000))
    model.fit(texts, labels)
    return model

def detect_priority(text: str) -> str:
    words = set(preprocess(text).split())
    if words & URGENT_WORDS:
        return "High"
    return "Medium" if len(words) > 12 else "Low"
```

```

def classify(model, text: str):
    """Return (category, confidence)."""
    probs = model.predict_proba([preprocess(text)])[0]
    best = probs.argmax()
    return str(model.classes_[best]), float(probs[best])

def process_ticket(model, ticket_id: int, text: str) -> dict:
    """Run one ticket through the full flow."""
    category, confidence = classify(model, text)
    priority = detect_priority(text)
    if confidence >= CONFIDENCE_THRESHOLD:
        route, status = TEAM_ROUTES[category], "Auto-routed"
    else:
        route, status = "Manual Review Queue", "Needs human review"
    return {"id": ticket_id, "category": category, "confidence": round(confidence, 2),
            "priority": priority, "route": route, "status": status}

def notify(result: dict) -> None:
    print(f"[ACK] Ticket #{result['id']} received.")
    if result["priority"] == "High":
        print(f"[ALERT] High priority ticket #{result['id']} sent to {result['route']}")

if __name__ == "__main__":
    model = build_model()
    samples = ["My app crashed immediately after payment, please fix asap",
               "I was charged twice this month",
               "Cannot login, I forgot my password",
               "Tell me about your office timings"]
    for i, text in enumerate(samples, start=1):
        result = process_ticket(model, i, text)
        notify(result)
        print(result, "\n")

```

Sample Output

[ACK] Ticket #1 received.

[ALERT] High priority ticket #1 sent to Manual Review Queue

```
{'id': 1, 'category': 'Account', 'confidence': 0.37, 'priority': 'High', 'route': 'Manual Review Queue', 'status': 'Ne
```

[ACK] Ticket #2 received.

```
{'id': 2, 'category': 'Billing', 'confidence': 0.68, 'priority': 'Low', 'route': 'Billing Team', 'status': 'Auto-rout
```

[ACK] Ticket #3 received.

```
{'id': 3, 'category': 'Account', 'confidence': 0.67, 'priority': 'Low', 'route': 'Account Services Team', 'status': 'Ne
```

[ACK] Ticket #4 received.

```
{'id': 4, 'category': 'General', 'confidence': 0.46, 'priority': 'Low', 'route': 'Manual Review Queue', 'status': 'Ne
```

Explanation

- Ticket 2 and 3 are clear cases: high confidence, so they are auto-routed to the Billing and Account teams.
- Ticket 1 mixes crash and payment words, so confidence is low and it goes to the Manual Review Queue, while the alert still marks it High priority.
- The threshold (0.50) can be tuned. A larger, real ticket dataset will improve confidence and accuracy.